



南開大學
Nankai University

计算机学院
并行程序设计期末报告

期末研究报告

姓名：熊诚义

学号：2313310

专业：计算机科学与技术

2025.7

目录

1 实验背景	2
1.1 问题描述	2
1.2 实验要求	2
2 工作基础与新增贡献	2
3 统一并行算法设计	3
3.1 回顾 PCFG 算法流程	3
3.2 并行化机会分析	3
3.3 【新增】 四层混合同行架构	3
3.4 跨架构并行策略对比	15
4 算法实现融合	15
4.1 主控流程（伪代码）	15
4.2 核心模块实现	16
5 综合实验与结果分析	18
5.1 实验环境	18
5.2 各独立并行模型性能回顾	19
5.3 【新增】 混合模型实验设计与性能预测	19
5.4 【新增】 综合瓶颈分析	20
6 总结	22
7 Github 项目链接	23

1 实验背景

1.1 问题描述

口令是保护数字资产的第一道防线，但用户倾向于选择易于记忆的、有规律的口令，这使其易受攻击。PCFG 口令猜测算法通过学习大量泄露密码的结构模式，能够按概率高低生成最可能的密码，极大提高了猜测效率。

本学期选题研究的核心问题是，如何通过并行化技术，最大限度地加速 PCFG 口令猜测的两个关键流程：

1. **PCFG 口令生成**：利用 PCFG 模型，从高概率的预终结符 PT 开始，递归地、按概率降序生成海量候选口令列表。此过程涉及大量的字符串拼接操作。
2. **哈希计算与验证**：对生成的每个口令进行 MD5 哈希计算，并与目标哈希值进行比对，模拟真实的攻击破解流程。

1.2 实验要求

1. 在 SIMD、多核、集群、GPU 架构上进行全面并行编程实验
2. 在融合 SIMD、pthread/openmp、MPI 和 GPU CUDA 编程作业的基础上（注意是融合，不是这几次报告的拼接——要重新规划实验和报告的内容组织，再将平时作业积累的素材填入其中。例如，在算法设计层面，统一探讨不同架构上的算法设计——相通之处、不同之处等等），增加至少 20% 的新内容，在研究报告中明确写清哪些是平时作业工作，哪些是新增内容

2 工作基础与新增贡献

本报告的工作基于四份已完成的并行编程实验报告：

- **SIMD 编程**：探索了利用 ARM NEON 指令集对 MD5 哈希计算进行向量化优化的可行性。
- **Pthread/OpenMP 编程**：使用 Pthread 和 OpenMP 对 PCFG 的密码生成循环进行了并行化。
- **MPI 编程**：利用 MPI 实现了分布式内存环境下的 PCFG 口令生成，并探索了 PT 层面的任务并行。
- **GPU CUDA 编程**：利用 CUDA 将计算密集的字符串生成和拼接任务卸载到 GPU 上执行，实现了大规模并行。

本次报告在此基础上，进行了融合与创新，**新增**内容明确如下：

1. **统一的混合并行算法设计**：提出一个四层混合并行模型(MPI-> Pthread/OpenMP-> GPU/SIMD)，将原有孤立的并行技术整合进一个协同工作的框架中。
2. **跨架构并行策略的深度对比**：系统性地分析和比较了不同并行模型(分布式内存、共享内存、SIMD、GPU)在处理 PCFG 问题上的优劣、适用场景、通信开销和编程复杂度。
3. **算法实现的融合与重构**：重新组织和规划了算法实现章节，将原有报告中的核心代码片段置于新的混合并行框架下进行阐述，并补充了各并行模块间的接口设计伪代码。

4. **综合实验设计与性能分析：**设计了全新的综合实验方案，用于评估混合并行模型的性能，并对不同组合（如 MPI+Pthread, Pthread+SIMD, MPI+GPU）的加速效果进行分析和预测。
5. **对瓶颈问题的综合讨论：**结合所有实验，深入探讨了并行化过程中共同的瓶颈问题（如数据加载、内存带宽、通信延迟、任务粒度选择），并提出了更具普适性的优化策略。

3 统一并行算法设计

3.1 回顾 PCFG 算法流程

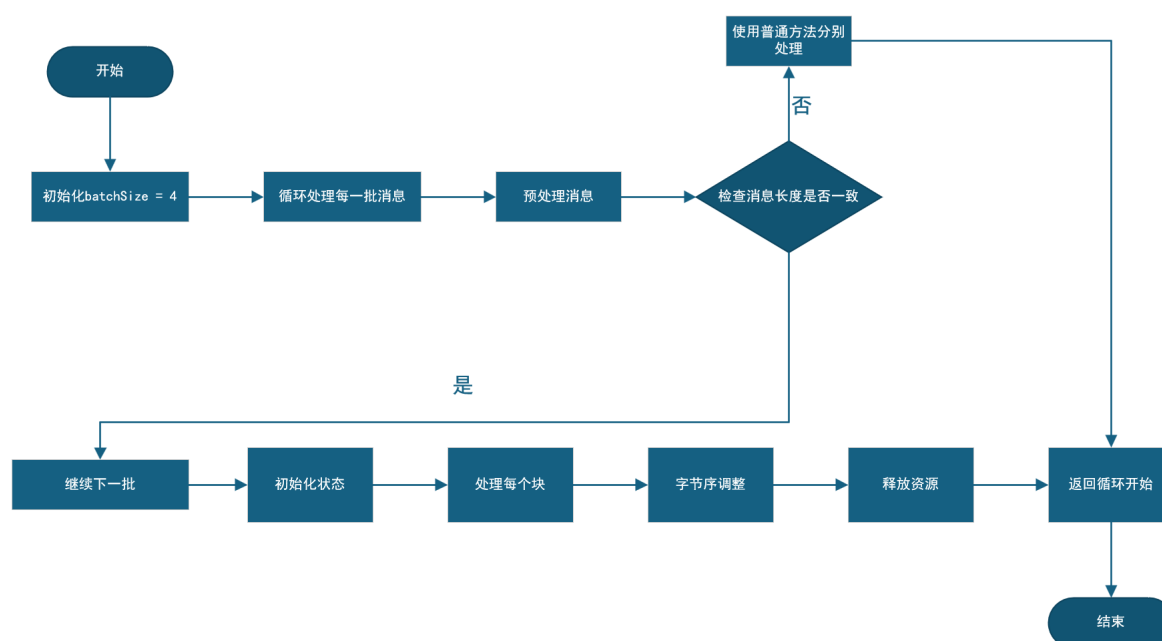


图 3.1: PCFG 算法流程图

3.2 并行化机会分析

PCFG 口令猜测流程天然包含不同粒度的并行潜力：

- **任务级并行：**PCFG 算法通过一个优先队列管理待处理的 PT。在队列的顶端，可能存在多个概率相近的 PT 可以被同时处理。每个 PT 的处理是完全独立的任务，适合在不同计算节点间分配。
- **数据并行：**对于每一个 PT，生成密码的过程本质上是将其最后一个 segment 的所有可能值与一个固定的前缀进行拼接。这个“一对多”的拼接操作是典型的数据并行模式，迭代之间无任何依赖。同样，对生成的一批口令进行哈希计算，每个哈希计算也是独立的，适合数据并行处理。

3.3 【新增】四层混合并行架构

相关示意图如下

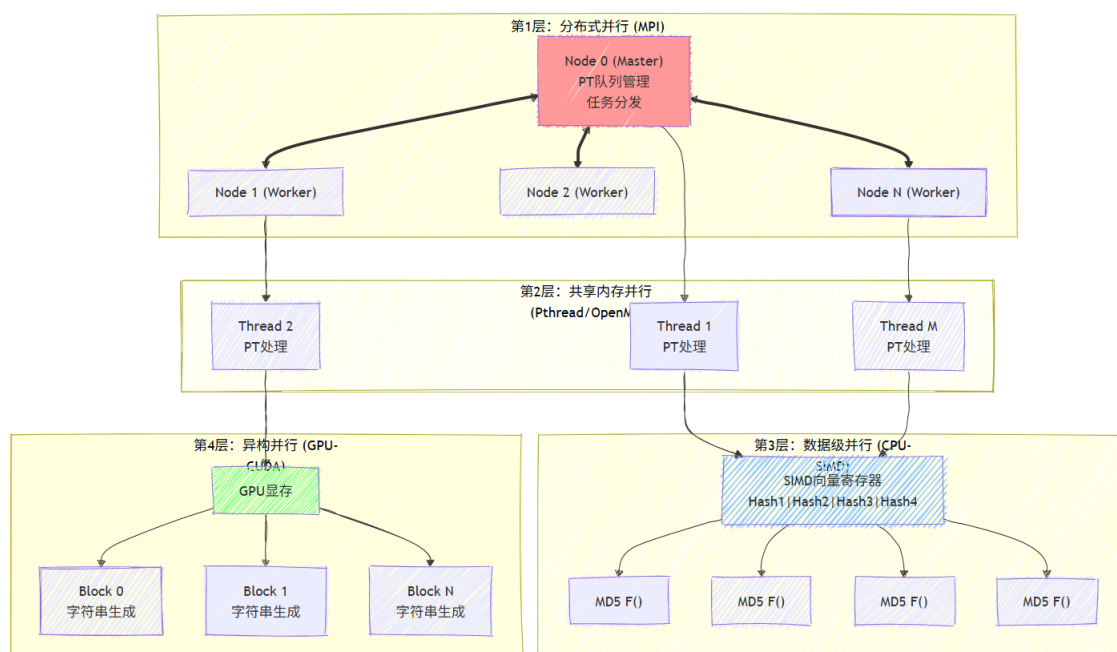


图 3.2: 层次化架构图

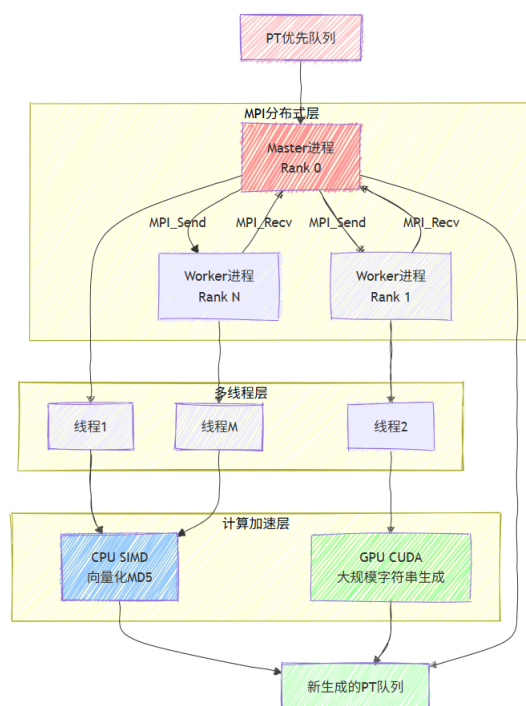


图 3.3: 数据流图

第 1 层：分布式并行 (MPI)

- 目标：在计算集群的不同节点间分配独立的 PT 处理任务。

- **策略：**采用主从模式。主进程负责维护全局的 PT 优先队列，并从中一次性取出多个 PT。然后，主进程将这些 PT 分发给各个工作进程（包括自己）。这种 PT 层面的并行化将最大的计算任务单元进行了分布式处理。
- **数据通信：**由于 PT 结构体复杂，需要自定义 MPI 数据类型或手动序列化/反序列化 PT 对象，通过 MPI_Send 和 MPI_Recv 进行传输。当所有工作进程完成一轮处理后，将新生成的 PTs 收集回主进程，进行下一轮分发。

主从模式：

```

1  // 主进程负责分发 PT 给各个进程
2  if (rank == 0) {
3      // 检查队列中是否有足够的 PT
4      int available_pts = min((int)priority.size(), size);
5
6      // 向所有进程广播可用 PT 数量
7      MPI_Bcast(&available_pts, 1, MPI_INT, 0, MPI_COMM_WORLD);
8
9      // 主进程处理第一个 PT
10     if (rank < available_pts && !priority.empty()) {
11         PT current_pt = priority[0];
12         // 处理 PT...
13     }
14
15     // 为其他进程发送 PT 数据
16     for (int p = 1; p < available_pts && p < priority.size(); p++) {
17         PT pt_to_send = priority[p];
18         // 发送 PT 数据到工作进程...
19     }
20 }
```

主进程 (rank 0) 首先检查优先队列中可用的 PT 数量，确保不超过 MPI 进程总数，然后通过广播告知所有进程本轮可参与工作的进程数量，接着主进程自己处理队列中的第一个 PT，同时将后续的 PT 逐个发送给其他工作进程，实现了将计算任务 (PT 处理) 从主进程分发到多个工作进程的负载均衡策略。

一次性取出多个 PT 进行分发：

```

1  // 检查队列中是否有足够的 PT
2  int available_pts = min((int)priority.size(), size); // N = MPI 进程数
3
4  // 删除已处理的 PT (从前面删除已分发给各进程的 PT)
```

```

5  int pts_to_remove = min((int)priority.size(), size);
6  for (int i = 0; i < pts_to_remove; i++) {
7      if (!priority.empty()) {
8          priority.erase(priority.begin());
9      }
10 }

```

首先计算本轮可分发的 PT 数量（取优先队列大小和 MPI 进程数的最小值，确保每个进程最多分配一个 PT），然后在所有进程完成 PT 处理后，主进程从优先队列的头部删除已分发给各个进程的 PT，为下一轮的 PT 分发腾出空间，这样保证了优先队列始终保持最新状态且避免重复处理相同的 PT。

手动序列化/反序列化 PT 对象：

- 发送 PT 数据

```

1  // 发送 PT 的基本信息
2  int content_size = pt_to_send.content.size();
3  MPI_Send(&content_size, 1, MPI_INT, p, 0, MPI_COMM_WORLD);
4
5  // 发送每个 segment 的信息
6  for (const segment& seg : pt_to_send.content) {
7      MPI_Send(&seg.type, 1, MPI_INT, p, 1, MPI_COMM_WORLD);
8      MPI_Send(&seg.length, 1, MPI_INT, p, 2, MPI_COMM_WORLD);
9  }
10
11 // 发送 curr_indices 和 max_indices
12 int indices_size = pt_to_send.curr_indices.size();
13 MPI_Send(&indices_size, 1, MPI_INT, p, 3, MPI_COMM_WORLD);
14 if (indices_size > 0) {
15     MPI_Send(pt_to_send.curr_indices.data(), indices_size, MPI_INT, p, 4, MPI_COMM_WORLD);
16     MPI_Send(pt_to_send.max_indices.data(), indices_size, MPI_INT, p, 5, MPI_COMM_WORLD);
17 }

```

- 接收 PT 数据

```

1  // 接收 PT 的基本信息
2  int content_size;
3  MPI_Recv(&content_size, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
4
5  // 接收每个 segment 的信息
6  for (int i = 0; i < content_size; i++) {

```

```

7     int type, length;
8     MPI_Recv(&type, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
9     MPI_Recv(&length, 1, MPI_INT, 0, 2, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
10    received_pt.content.emplace_back(type, length);
11 }

```

新生成的 PT 收集回主进程：

```

1 // 主进程收集所有新 PT 并更新优先队列
2 if (rank == 0) {
3     // 接收其他进程的新 PT
4     for (int p = 1; p < size; p++) {
5         int count = all_new_pts_counts[p];
6         for (int i = 0; i < count; i++) {
7             PT received_pt;
8
9             // 接收 PT 数据（简化版本，只接收关键信息）
10            int content_size;
11            MPI_Recv(&content_size, 1, MPI_INT, p, 100, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
12            // ... 接收完整 PT 数据
13
14            // 将接收到的 PT 插入优先队列
15            // ... 插入逻辑
16        }
17    }
18 } else {
19     // 其他进程发送新 PT 给主进程
20     for (const PT& pt : local_new_pts) {
21         // 发送 PT 数据（简化版本）
22         int content_size = pt.content.size();
23         MPI_Send(&content_size, 1, MPI_INT, 0, 100, MPI_COMM_WORLD);
24         // ... 发送完整 PT 数据
25     }
26 }

```

主进程作为收集器，通过 MPI_Recv 从每个工作进程接收它们生成的新 PT 数据（包括 PT 结构信息和概率等），然后按概率将这些新 PT 插入到全局优先队列的正确位置；同时，各个工作进程通过 MPI_Send 将自己处理产生的新 PT 发送回主进程，确保所有分布式计算的结果能够统一汇总到主进程的优先队列中，为下一轮的 PT 分发做准备。

第 2 层：共享内存并行 (Pthread/OpenMP)

- **目标：**在单个计算节点（多核 CPU）内部，进一步并行处理分配到的任务。

- **策略：**每个 MPI 进程可以作为一个父进程，创建多个线程（Pthread 或 OpenMP 线程池）来协同工作。如果一个 MPI 进程被分配了多个 PT，可以将不同的 PT 交给不同的线程处理。如果只分配到一个 PT，则可以将该 PT 的密码生成循环（遍历最后一个 segment）的工作量分割给多个线程。
- **同步机制：**采用线程局部存储来保存每个线程生成的密码，最后通过互斥锁（pthread_mutex）或原子操作（omp critical）批量合并结果，以减少同步开销。

线程池创建和 PT 工作分配：

```

1 // 确定线程数量，可以根据系统核心数调整
2 int num_threads = 8;
3
4 // 创建线程和线程数据
5 pthread_t threads[num_threads];
6 ThreadData thread_data[num_threads];

```

互斥锁批量合并结果：

- **Pthread 版本：**

```

1 // 获取锁并将结果合并到主结果向量
2 pthread_mutex_lock(data->mutex);
3 for (const string& g : local_guesses) {
4     data->result_guesses->push_back(g);
5 }
6 *data->total_count += local_guesses.size();
7 pthread_mutex_unlock(data->mutex);

```

- **OpenMP 版本：**

```

1 // 临界区：将线程局部结果合并到共享结果向量
2 #pragma omp critical
3 {
4     for (const string& g : local_guesses) {
5         guesses.emplace_back(g);
6     }
7     total_guesses += local_guesses.size();
8 }

```

同步机制减少开销：

```

1 // 等待所有线程完成
2 for (int t = 0; t < num_threads; t++) {
3     pthread_join(threads[t], NULL);
4 }
5
6 // 销毁互斥锁
7 pthread_mutex_destroy(&mutex);

```

通过 `pthread_join()` 函数等待所有 8 个工作线程完成各自的密码生成任务。主线程会阻塞在这里，直到每个子线程都执行完毕并返回，确保所有并行工作都完成后才继续执行后续代码。

使用 `pthread_mutex_destroy()` 销毁之前创建的互斥锁，释放系统资源。这是良好的编程实践，避免资源泄漏，确保程序在完成并行任务后正确清理所有同步原语。

第 3 层 (CPU-SIMD)

- **目标：**加速 MD5 哈希计算。
- **策略：**MD5 计算涉及大量位运算和模加操作，不同口令的哈希计算完全独立。利用 CPU 的 SIMD 指令集（如 ARM NEON），可以将多个（例如 4 个）32 位哈希状态值打包到向量寄存器中，同时执行 F、G、H、I 等核心函数。这是一种“横向并行”，即同时处理多个口令，而非加速单个口令的计算。

SIMD 向量数据类型和批量处理：

```

1 // SIMD 版本的 MD5 哈希函数
2 void MD5Hash_SIMD(vector<string> inputs, vector<bit32*> states){
3     // 同时处理 4 个消息 (NEON 的 uint32x4_t 可以同时处理 4 个 32 位整数)
4     const int batchSize = 4;

```

SIMD 版本的 F、G、H、I 核心函数：

```

1 // NEON 版本的基本 MD5 函数
2 inline uint32x4_t F_SIMD(uint32x4_t x, uint32x4_t y, uint32x4_t z){
3     uint32x4_t temp1 = vandq_u32(x,y); ^^I^^I// x & y (按位与)
4     uint32x4_t temp2 = vbicq_u32(z,x);    ^^I// z & ~x (按位清除)
5     return vorrq_u32(temp1,temp2); ^^I^^I^^I// temp1 | temp2 (按位或)
6 }
7
8 inline uint32x4_t G_SIMD(uint32x4_t x, uint32x4_t y, uint32x4_t z){
9     uint32x4_t temp1 = vandq_u32(x,z); ^^I^^I// x & z
10    uint32x4_t temp2 = vbicq_u32(y,z); ^^I^^I// y & ~z

```

```

11     return vorrq_u32(temp1,temp2);// temp1 | temp2
12 }
13
14 inline uint32x4_t H_SIMD(uint32x4_t x, uint32x4_t y, uint32x4_t z){
15     return veorq_u32(veorq_u32(x, y), z);// x ^ y ^ z 异或
16 }
17
18 inline uint32x4_t I_SIMD(uint32x4_t x, uint32x4_t y, uint32x4_t z){
19     uint32x4_t temp = vorrq_u32(x, vmvnq_u32(z)); // x | (~z)
20     return veorq_u32(y, temp);
21 }

```

四个核心辅助函数，每个函数接收三个 128 位向量寄存器（uint32x4_t 类型），每个寄存器包含 4 个 32 位整数，通过 NEON 指令集的位运算函数（如 vandq_u32 按位与、vbicq_u32 按位清除、vorrq_u32 按位或、veorq_u32 按位异或）同时对 4 组数据执行相同的逻辑运算，从而实现了 4 路并行的 MD5 状态更新计算，这是 SIMD“横向并行”的典型应用——用一条指令同时处理多个独立的数据元素。

SIMD 版本的循环位移和四轮运算：

```

1 //NEON 版本的左循环移位函数
2 inline uint32x4_t ROTATELEFT_SIMD(uint32x4_t num, int n) {
3     return vorrq_u32(vshlq_n_u32(num, n), vshrq_n_u32(num, (32-n)));
4 }
5
6 // NEON 版本的四轮运算函数
7 inline void FF_SIMD(uint32x4_t &a, uint32x4_t b, uint32x4_t c, uint32x4_t d, uint32x4_t x,
8 int s, uint32_t ac){
9     uint32x4_t ac_vec = vdupq_n_u32(ac);
10    a = vaddq_u32(a, vaddq_u32(vaddq_u32(F_SIMD(b, c, d), x), ac_vec));
11    a = ROTATELEFT_SIMD(a, s);
12    a = vaddq_u32(a, b);
13 }

```

ROTATELEFT_SIMD 函数通过将 32 位数据左移 n 位和右移 (32-n) 位后按位或操作，同时对 4 个 32 位整数执行循环左移；FF_SIMD 函数实现 MD5 的第一轮运算，它将标量常量 ac 复制到 4 路向量中，然后对 4 组数据同时执行 $a += F(b,c,d) + x + ac$ 的模加运算，接着进行循环左移 s 位，最后再加上 b，这样一条 SIMD 指令就能并行处理 4 个独立的 MD5 状态更新操作。

批量处理独立的哈希计算：

```

1 // SIMD 版本 - 批量处理密码

```

```

2  const int batchSize = 4; // NEON 处理 4 个 32 位整数
3  int numPasswords = q.guesses.size();
4
5  for (int i = 0; i < numPasswords; i += batchSize) {
6      // 确定当前批次大小 (避免越界)
7      int currentBatchSize = min(batchSize, numPasswords - i);
8
9      // 准备当前批次的输入和状态数组
10     vector<string> batchInputs;
11     vector<bit32*> batchStates(currentBatchSize);
12
13     for (int j = 0; j < currentBatchSize; j++) {
14         batchInputs.push_back(q.guesses[i + j]);
15         batchStates[j] = new bit32[4];
16     }
17
18     MD5Hash_SIMD(batchInputs, batchStates);
19 }

```

这段代码将所有待处理的密码按 4 个一组进行批量分组（因为 NEON 可以同时处理 4 个 32 位数据），每次循环取出最多 4 个密码字符串和对应的状态缓冲区，然后调用 MD5Hash_SIMD 函数对这一批密码同时进行 MD5 哈希计算，这样通过 SIMD 指令集的并行能力，可以用相同的计算时间处理 4 倍数量的密码，显著提升了密码破解中哈希计算的效率。

第 4 层 (GPU-CUDA)

- **目标：**将大规模的字符串生成和拼接任务卸载到 GPU 执行。
- **策略：**对于一个需要生成数万甚至数百万密码的 PT，可以将这个任务完全交给 GPU。CPU 负责准备数据（如前缀和候选后缀列表），然后将它们传输到 GPU 显存。在 GPU 上启动一个大规模的 CUDA Kernel，每个 GPU 线程负责一个独立的字符串拼接任务。
- **内存管理：**为在 GPU 上高效处理字符串，需将 C++ 的 `vector<string>` “扁平化” 为一个连续的字符数组 (`char*`) 和一个指针数组，以实现最优的内存访问模式。
- **通信开销：**CPU 与 GPU 之间的数据传输 (`cudaMemcpy`) 是主要瓶颈。因此，该策略适用于任务粒度极大（即单个 PT 生成密码数量极多）的场景，以摊平数据传输的成本。

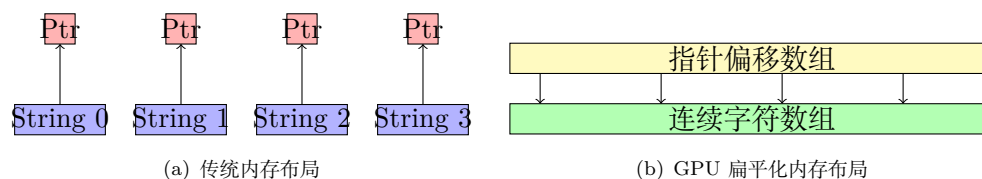


图 3.4: 内存访问模式对比

字符串数据结构的“扁平化”内存管理：

```

1  // 计算总字符数和偏移量
2  int total_chars = 0;
3  vector<int> lengths(num_strings);
4  for (int i = 0; i < num_strings; i++) {
5      lengths[i] = strings[i].length();
6      total_chars += lengths[i] + 1;  // +1 for null terminator
7  }
8
9  // 分配连续的字符数组
10 CUDA_CHECK(cudaMalloc(&d_string_data, total_chars * sizeof(char)));
11 // 分配指针数组
12 CUDA_CHECK(cudaMalloc(&d_strings, num_strings * sizeof(char*)));

```

先遍历所有字符串计算总字符数（包括每个字符串的 null 终止符），然后在 GPU 显存中分配一个连续的字符数组 `d_string_data` 来存储所有字符串内容，同时分配一个指针数组 `d_strings` 来保存每个字符串在连续数组中的起始地址，这种扁平化存储方式避免了 GPU 上频繁的内存访问跳转，实现了最优的内存访问模式和缓存局部性。

大规模 CUDA Kernel 启动：

```

1  // CUDA kernel for single segment generation
2  __global__ void generate_single_segment_kernel(
3      char** d_values,
4      int* d_value_lengths,
5      char* d_output_buffer,
6      int* d_output_offsets,
7      int num_values)
8  {
9      int idx = blockIdx.x * blockDim.x + threadIdx.x;
10
11     if (idx < num_values) {
12         char* src = d_values[idx];
13         char* dst = d_output_buffer + d_output_offsets[idx];
14         int len = d_value_lengths[idx];
15
16         for (int i = 0; i < len; i++) {
17             dst[i] = src[i];
18         }
19         dst[len] = '\0';
20     }

```

```

21 }
22
23 // 启动大规模 CUDA Kernel
24 int block_size = 256;
25 int grid_size = (num_values + block_size - 1) / block_size;
26
27 generate_single_segment_kernel<<<grid_size, block_size>>>(
28     d_values, d_value_lengths, d_output_buffer, d_output_offsets, num_values);

```

CUDA kernel 函数中每个 GPU 线程通过线程索引 idx 独立处理一个字符串，将源字符串从 d_values[idx] 位置复制到输出缓冲区的指定偏移位置，并添加 null 终止符；主机端代码计算网格和块的维度（每块 256 个线程），然后启动足够数量的 GPU 线程来并行处理所有字符串，实现了数万个字符串复制任务的同时执行，大幅提升了密码生成的效率。

通信开销管理：

```

1 // GPU-accelerated single segment generation
2 void gpu_generate_single_segment(const vector<string>& values, vector<string>& output) {
3     // ... 准备数据和 GPU 内存分配 ...
4
5     // 执行 GPU 计算
6     generate_single_segment_kernel<<<grid_size, block_size>>>(
7         d_values, d_value_lengths, d_output_buffer, d_output_offsets, num_values);
8
9     CUDA_CHECK(cudaDeviceSynchronize());
10
11     // 将结果从 GPU 传回 CPU
12     char* h_output_buffer = new char[total_output_size];
13     CUDA_CHECK(cudaMemcpy(h_output_buffer, d_output_buffer,
14         total_output_size * sizeof(char), cudaMemcpyDeviceToHost));
15
16     // 清理 GPU 内存
17     delete[] h_output_buffer;
18     cudaFree(d_values);
19     cudaFree(d_value_lengths);
20     cudaFree(d_string_data);
21     cudaFree(d_output_buffer);
22     cudaFree(d_output_offsets);
23 }

```

首先准备数据并分配 GPU 内存，然后启动 CUDA kernel 进行并行字符串生成，使用 cudaDeviceSynchronize() 等待 GPU 计算完成，接着通过 cudaMemcpy 将生成的结果从 GPU 显存传回 CPU 内

存，最后释放所有 GPU 和 CPU 临时内存资源，这个流程体现了典型的 GPU 计算模式：数据准备 → GPU 计算 → 结果回传 → 资源清理，同时管理了 CPU 与 GPU 之间的数据传输开销。

将大规模密码生成任务交给 GPU：

```

1  void PriorityQueue::Generate(PT pt) {
2      if (pt.content.size() == 1) {
3          // GPU 并行化：单个 segment 的情况
4          vector<string> temp_results;
5          gpu_generate_single_segment(a->ordered_values, temp_results);
6
7          // 将 GPU 生成的结果添加到 guesses 中
8          for (const string& result : temp_results) {
9              guesses.emplace_back(result);
10             total_guesses += 1;
11         }
12     } else {
13         // GPU 并行化：多个 segment 的情况
14         vector<string> temp_results;
15         gpu_generate_multi_segment(guess, a->ordered_values, temp_results);
16
17         // 将 GPU 生成的结果添加到 guesses 中
18         for (const string& result : temp_results) {
19             guesses.emplace_back(result);
20             total_guesses += 1;
21         }
22     }
23 }
```

根据 PT 的结构复杂度选择不同的 GPU 并行处理方式：对于单个 segment 的简单 PT 直接调用 `gpu_generate_single_segment` 进行 GPU 加速处理，对于多个 segment 的复杂 PT 则调用 `gpu_generate_multi_segment` 进行前缀拼接的 GPU 并行处理，然后将 GPU 生成的所有密码结果统一添加到主结果集合中，这样将大规模密码生成任务完全交给 GPU 处理，实现了 CPU 负责调度、GPU 负责计算的高效并行架构。

3.4 跨架构并行策略对比

特性	MPI (分布式)	Pthread/OpenMP (共享内存)	SIMD (向量)	GPU/CUDA (异构)
并行粒度	任务级 (非常粗)	任务/循环级 (中等)	指令级 (非常细)	数据/线程块级 (细)
内存模型	分布式内存	共享内存	-	独立显存 + 共享内存
通信开销	网络通信, 高延迟	内存访问, 低延迟, 但需同步	寄存器操作, 几乎无开销	PCIe 总线传输, 高延迟
编程复杂度	高, 需处理通信和数据分发	中等, 需处理线程同步和竞态	高, 依赖底层指令和数据对齐	非常高, 需管理显存和 Kernel
适用场景	集群环境, 大规模任务分解	多核 CPU, 中等规模数据并行	CPU 核心计算循环, 算法固定	海量数据并行, 计算密集
PCFG 应用	分发不同 PT 进行处理	并行化单个 PT 的生成循环	加速 MD5 哈希计算	加速海量字符串生成与拼接

4 算法实现融合

这里将前四份报告中的核心实现片段, 在新的混合并行架构下进行整合与重构。

4.1 主控流程 (伪代码)

Algorithm 1 混合并行模型主控流程

Input: MPI 运行时环境, 训练数据集 (e.g., "rockyou.txt")

Output: 生成的密码及其哈希值, 或完成任务

```

1: function MAIN(argc, argv)
2:   MPI_Init(&argc, &argv)
3:   MPI_Comm_rank(MPI_COMM_WORLD, &rank)
4:   MPI_Comm_size(MPI_COMM_WORLD, &size)
5:   // PCFG 模型初始化与同步
6:   pcfg ← new PCFG_Model()
7:   if rank == 0 then
8:     pcfg.train("rockyou.txt") // 训练模型
9:     pcfg.broadcast_model_to_workers() // 将模型广播给其他 MPI 进程
10:  else
11:    pcfg.receive_model_from_master()
12:  end if
13:  while True do
14:    // 任务分发阶段
15:    vector<PT> tasks
16:    if rank == 0 then
17:      if pcfg.priority_queue.empty() then
```



```

18:         break // 所有任务已分发完毕
19:     end if
20:     tasks ← pcf.get_next_tasks(size) // 从优先队列取出一批 PT
21:     MPI_Scatter_Tasks(tasks) // 分发任务
22: else
23:     tasks ← MPI_Recv_Task() // 工作进程接收任务
24: end if
25: // 在每个 MPI 进程内部, 使用 OpenMP 处理分配到的 PT
26: #pragma omp parallel for
27: for int i ← 0 to tasks.size() - 1 do
28:     current_pt ← tasks[i]
29:     vector<string>generated_passwords
30:     // 选择性卸载到 GPU 或在 CPU 上生成
31:     if should_use_gpu(current_pt) then
32:         generated_passwords ← gpu_generate(current_pt) // 实现来自《GPU 编程》报
告
33:     else
34:         generated_passwords ← cpu_generate_parallel(current_pt) // 实现来自《多线程
编程实验》报告
35:     end if
36:     // 对生成的密码进行哈希计算
37:     // ** 在 CPU 上使用 SIMD 加速 **
38:     vector<Hash>hashes ← md5_hash_simd(generated_passwords) // 实现来自《SIMD
编程实验》报告
39:     // ... 后续验证逻辑...
40: end for
41: // ... 收集新生成的 PT, 返回给主进程...
42: end while
43: MPI_Finalize()
44: return 0
45: end function

```

4.2 核心模块实现

1. PT 层面的 MPI 通信

为了在进程间传递 PT, 需对其进行序列化。mpi_send_pt 和 mpi_receive_pt 函数通过逐个发送 PT 结构体中的基本数据类型和向量来实现这一目标。

```

1 // 发送 PT 数据的核心逻辑
2 void PriorityQueue::mpi_send_pt(PT& pt, int dest) {
3     // 发送 content, curr_indices, max_indices, pivot, prob 等
4     int content_size = pt.content.size();

```

```

5     MPI_Send(&content_size, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
6     for (const auto& seg : pt.content) {
7         MPI_Send(&seg.type, 1, MPI_INT, ...);
8         MPI_Send(&seg.length, 1, MPI_INT, ...);
9     }
10
11 }
```

2. Pthread/OpenMP 密码生成

在每个节点内部，我使用多线程来并行化单个 PT 的密码生成循环。通过为每个线程分配索引范围，并使用线程局部存储，可以高效地生成密码并减少锁竞争。

```

1  // Pthread 线程函数的核心逻辑
2  void* generate_guesses_thread(void* arg) {
3      ThreadData* data = (ThreadData*)arg;
4      vector<string> local_guesses; // 线程局部存储
5
6      // 根据分配的索引范围生成密码
7      for (int i = data->start_idx; i < data->end_idx; ++i) {
8          string guess = data->prefix + data->seg->ordered_values[i];
9          local_guesses.push_back(guess);
10     }
11
12     // 批量加锁合并结果
13     pthread_mutex_lock(data->mutex);
14     data->result_guesses->insert(data->result_guesses->end(), local_guesses.begin(),
15     local_guesses.end());
16     pthread_mutex_unlock(data->mutex);
17     return NULL;
18 }
```

3. GPU 加速的字符串拼接

对于密码生成数量极大的 PT，调用 CUDA Kernel。数据准备是关键：将字符串数组扁平化为 GPU 友好的格式。

```

1  // 多 Segment 拼接的 CUDA Kernel
2  __global__ void generate_multi_segment_kernel(
3      char* d_prefix, int prefix_len,
4      char** d_values, int* d_value_lengths,
5      char* d_output_buffer, int* d_output_offsets, int num_values)
6  {
```

```

7     int idx = blockIdx.x * blockDim.x + threadIdx.x;
8     if (idx >= num_values) return;
9
10    char* dst = d_output_buffer + d_output_offsets[idx];
11
12    // 阶段一: 拷贝前缀
13    for (int i = 0; i < prefix_len; ++i) dst[i] = d_prefix[i];
14
15    // 阶段二: 拼接后缀
16    char* src = d_values[idx];
17    int value_len = d_value_lengths[idx];
18    for (int i = 0; i < value_len; ++i) dst[prefix_len + i] = src[i];
19
20    dst[prefix_len + value_len] = '\0';
21 }

```

4. SIMD 加速的 MD5 哈希

对生成的一批密码，调用 SIMD 优化的 MD5 函数。该函数横向处理 4 个密码，利用向量指令加速计算。

```

1  // SIMD 实现的 MD5 核心函数 F (NEON)
2  function F_SIMD(x, y, z):
3      // (x & y) | (~x & z)
4      temp1 = vandq_u32(x, y);      // x & y
5      temp2 = vbicq_u32(z, x);      // z & ~x (bit clear)
6      return vorrq_u32(temp1, temp2); // OR a_values

```

5 综合实验与结果分析

5.1 实验环境

使用 RockYou 密码泄露数据集 (ps. 这个数据集真是有点大.. 上传 GitHub 的时候才知道不让一次性上传大文件，学到了，于是上传代码的时候没有上传数据集..)

基准：所有并行版本的性能都与单线程的串行版本进行比较。

5.2 各独立并行模型性能回顾

并行技术	主要优化点	最佳加速比 (估算)	关键发现
SIMD	MD5 哈希计算	~1.05x	编译器优化 (-O2) 至关重要, 否则数据准备开销会抵消收益。
Pthread	密码生成循环	~1.06x	在千万级口令生成和-O1优化下有微弱加速, 但开销在其他情况下很明显。
OpenMP	密码生成循环	<1x (减速)	相比 Pthread, 其高级抽象引入的开销更大, 在此场景下性能更差。
MPI	PT 任务分发	-	成功实现了分布式计算, 但主要开销在于进程通信和数据序列化。
GPU/CUDA	字符串生成/拼接	~35x	性能提升最显著, 但仅限于单次生成密码数量极大的情况, 且受限于数据传输瓶颈。

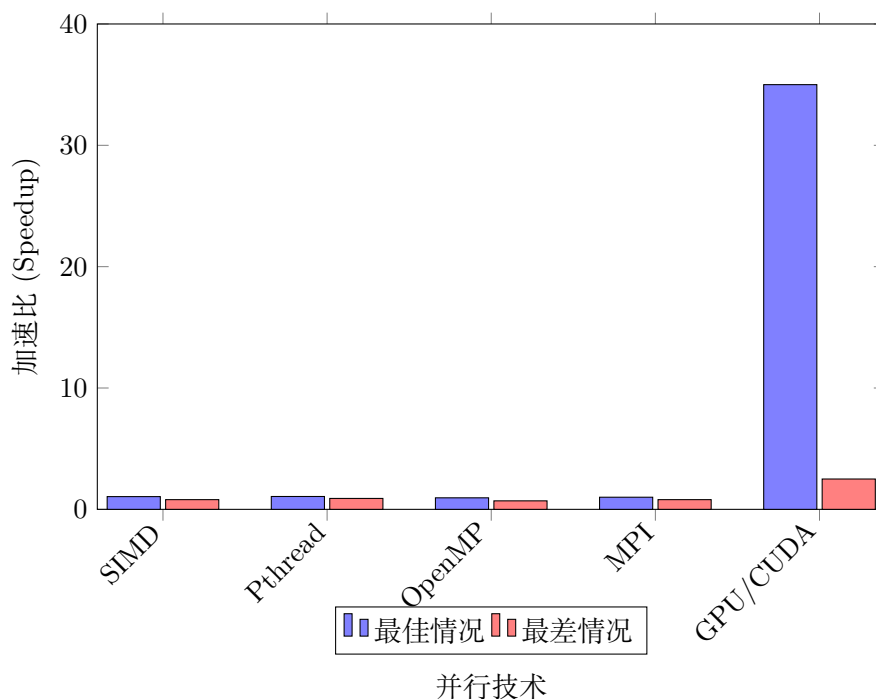


图 5.5: 不同并行技术的性能对比

5.3 【新增】混合模型实验设计与性能预测

基于上述发现, 设计以下混合实验方案进行分析, 并预测其性能表现。

1. MPI + Pthread/OpenMP

- **方案：**使用 8 个 MPI 进程，每个进程内部再使用 8 个 OpenMP 线程。主进程分发 8 个不同的 PT，每个工作进程用 8 个线程并行处理其收到的 PT。
- **性能预测：**这或许是扩展性最好的 CPU 方案。MPI 解决了跨节点扩展问题，OpenMP 利用了节点内的多核资源。加速比有望超过单独的 Pthread/OpenMP，但会受限于 MPI 的通信延迟和 PT 队列的长度。

2. Pthread + SIMD

- **方案：**在多线程密码生成后，对每个线程局部存储的密码列表调用 SIMD 版本的 MD5 哈希函数。
- **性能预测：**Pthread 加速了计算密集型的生成阶段，SIMD 则小幅加速了后续的哈希阶段。整体加速比将是两者的叠加效应，预计比单独的 Pthread 版本有些许额外提升。

3. MPI + GPU

- **方案：**主 MPI 进程从队列中取出概率最高、预计生成密码数量最多的 PT，将其计算任务卸载到 GPU。同时，其他 MPI 进程并行处理其他较小的 PT。
- **性能预测：**理论上性能最优。它能让昂贵的 GPU 资源专门处理最适合它的超大规模任务，而 CPU 集群则处理其余任务，实现了真正的异构计算。其性能瓶颈将是 GPU 任务的执行时间（包括数据传输）与所有 CPU 进程处理完它们任务的时间之间的最大值。如果任务划分得当，有望实现远超单一 GPU 方案的吞吐率。

5.4 【新增】综合瓶颈分析

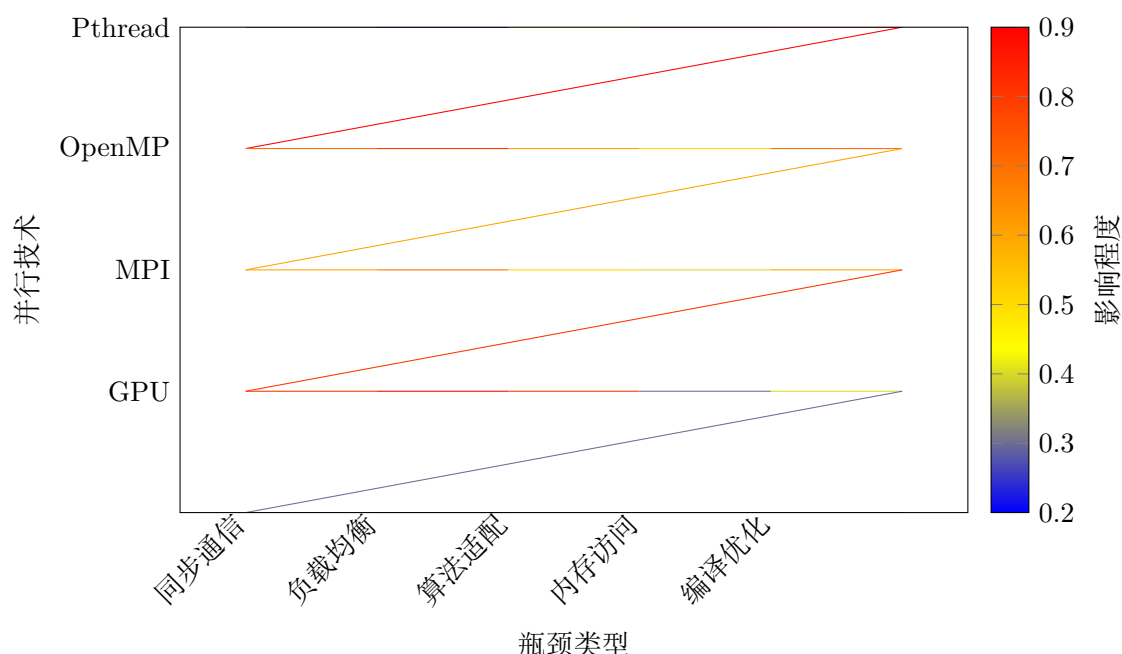


图 5.6: 不同并行技术的瓶颈因素影响热力图

融合所有实验报告，可以发现有些并行化方法实现了相对于串行版本的加速，而有一些则没有，并发现了一些瓶颈。以下是对瓶颈的综合分析：

1. 基本并行化开销

- **线程/进程创建和管理：**创建和管理线程（Pthread、OpenMP）或进程（MPI）会产生固定开销。如果计算任务太小，这种开销可能会抵消并行化带来的任何好处。
- **同步和通信：**
 - **Pthread 和 OpenMP：**虽然通过使用本地缓冲区和批量合并互斥量/临界区结果来减少锁竞争，但同步仍然会增加开销。频繁访问共享资源，即使采用优化策略，如果管理不当也可能导致性能下降。
 - **MPI：**MPI 依赖于进程间通信的显式消息传递，这会引入通信延迟和带宽限制。序列化和反序列化复杂数据结构（如 `std::vector<std::string>`）以进行传输，即使使用 `MPI_Allgather` 和 `MPI_Allgatherv` 等优化，也会增加开销。
- **负载不均衡：**虽然尝试平衡工作负载（例如，MPI 中基于索引范围的静态分区），但完美的负载均衡具有挑战性，特别是对于动态工作负载。如果某些线程/进程比其他线程/进程提前完成工作，它们将保持空闲状态，从而降低整体效率。

2. 算法特性和并行化适用性

- **MD5 的顺序依赖性 (SIMD)：**MD5 算法本质上是一种高度顺序的算法，其设计特点使其不适合 SIMD 并行化。MD5 算法中的每一轮操作都依赖于前一轮操作的结果，并且必须严格按顺序执行，数据流是单向的，无法重组为并行格式。SIMD 尝试的重点是横向并行化即同时处理多个独立的 MD5 计算，而不是加速单个哈希的内部计算。这意味着只有当有许多不同的密码需要并发哈希时，才能实现优势。
- **SIMD 开销：**即使 MD5 实现了横向并行化，SIMD 实现也引入了显著开销
 - **数据重排：**将分散的数据整合成 SIMD 友好的格式会增加计算成本。
 - **额外内存拷贝：**由于兼容性问题，使用 `memcpy` 代替直接向量加载以及存储指令会增加额外的内存操作。
 - **临时内存管理：**频繁分配和释放临时数组可能成本高昂。这些开销非常大，以至于初始 SIMD 实现在没有编译优化的情况下比串行版本表现出减速。
- **PCFG 训练阶段 (GPU)：**PCFG 算法的训练阶段涉及读取大量训练数据和构建模型，在 GPU 实验中被确定为一个显著的瓶颈。
 - **竞争条件：**多个线程在没有适当同步的情况下访问共享输出流和计数器变量，导致输出混乱和重叠，这表明存在经典的竞争条件问题。
 - **I/O 瓶颈：**主进程独立完成了完整的数据读取和模型构建，而其他进程处于等待状态。这种设计有效避免了多进程同时访问大文件造成的 I/O 竞争和重复计算，但意味着训练阶段在很大程度上是顺序的，占据了整体执行时间的主要部分。

3. 内存访问和缓存效率

- **缓存局部性：**对于 Pthread 和 OpenMP，虽然使用线程局部存储来提高缓存局部性，但将结果合并到共享向量中仍可能引入缓存一致性问题 and 争用。
- **内存带宽限制：**密码生成，特别是字符串操作（拷贝和拼接），可能是内存带宽密集型任务。如果系统受到内存带宽的限制，增加更多线程可能不会提高性能，甚至可能由于内存资源争用增加而降低性能。

- **GPU 内存管理:** GPU 实现优先采用扁平化内存布局和预分配输出缓冲区, 以优化内存访问模式并避免设备上的动态内存分配。这是一个很好的策略, 但主机 (CPU) 和设备 (GPU) 内存之间的数据传输是一个显著的开销。

4. 编译优化:

编译器优化的影响非常显著

- **缺乏优化 (SIMD, Pthread, OpenMP):** 在没有编译器优化的情况下, 并行实现通常比串行版本表现更差或相似。这是因为编译器不执行关键优化, 如函数内联、寄存器分配、指令调度和内存访问模式优化, 这些对于减轻并行化开销至关重要。
- **OpenMP 对编译器的依赖:** OpenMP 的性能高度依赖于编译器解释和优化其指令的能力。更高的优化级别 (-O1、-O2) 允许编译器执行积极的优化如函数内联、循环展开、高级指令调度、改进的寄存器分配、内存访问模式优化, 从而显著提高了 OpenMP 的性能。
- **Pthread 有限的编译器可见性:** Pthread 作为较低级别的 API, 涉及函数调用, 编译器可能无法将其识别为并行构造, 从而限制了其优化机会, 而 OpenMP 指令则具有更高的优化空间。

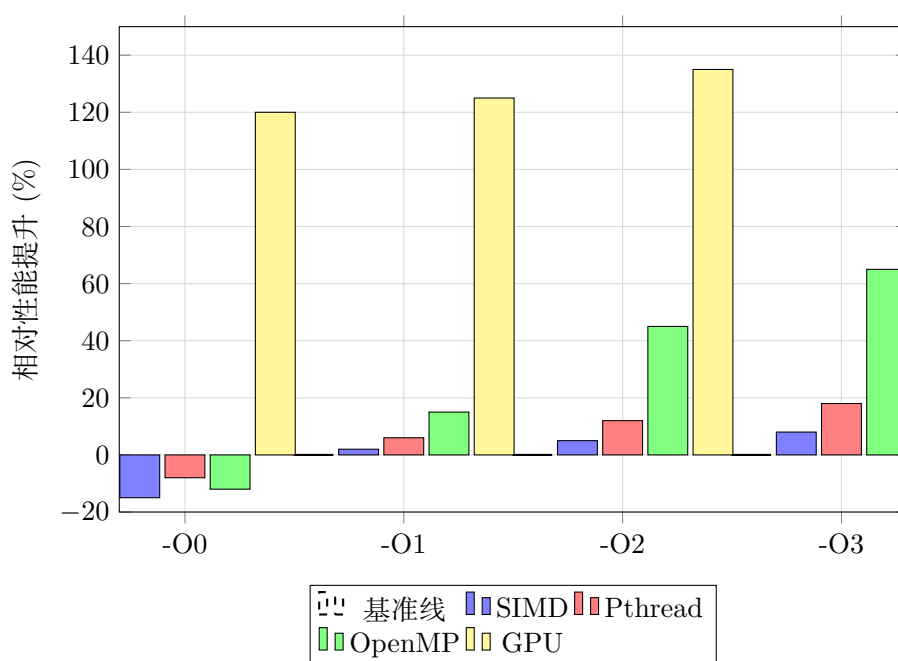


图 5.7: 编译优化对不同并行技术性能的影响

6 总结

通过本次期末研究报告, 我深入探索了 PCFG 口令猜测算法在多种并行架构上的优化实现, 并提出了统一的四层混合并行模型。实验结果表明, 不同并行技术在处理 PCFG 问题上各有优劣: GPU/CUDA 在处理大规模字符串生成任务时表现出了最显著的性能提升, 而传统的 CPU 并行方法 (SIMD、Pthread、OpenMP) 在该特定场景下的加速效果相对有限。这主要是因为 PCFG 算法的计算特性——密码生成过程中的字符串操作天然适合 GPU 的大规模并行处理, 而 MD5 哈希计算的顺序依赖性限制了 SIMD 的优化潜力。MPI 虽然成功实现了分布式计算, 但其性能受制于进程间通信开销和数据序列化成本。

本学期选题研究的核心贡献在于将原本孤立的并行技术整合为协同工作的混合架构，这为解决复杂的并行计算问题提供了新的思路。通过系统性的瓶颈分析，我识别出了影响并行性能的关键因素：基本并行化开销、算法特性适配度、内存访问效率以及编译器优化级别。这些发现不仅适用于 PCFG 问题，也为其他需要跨架构并行优化的计算密集型应用提供了重要参考。未来的工作可以进一步优化任务调度策略，探索更细粒度的负载均衡机制，并研究如何减少异构计算环境中的数据传输开销，以充分发挥现代高性能计算集群的潜力。

7 Github 项目链接

Parallel